

SIMATS ENGINEERING

ASSESSMENT TOOL 1-Pseudocode Writing Task

COURSE NAME: R PROGRAMMING

COURSE CODE: ITA04

COURSE OUTCOME COVERED

CO: Apply R programming fundamentals including data structures, vectors, matrices, arrays, and class types to perform mathematical operations and manage data effectively.
(Bloom Level -BL2- BL4)

Assessment Tool Weightage: 20%

QUESTIONS:

Total Marks: 100

1. Variables and Data Types Analysis (BL4 – Analyze)

Design pseudocode to simulate basic variable handling in R programming:

- Declare variables of different data types (numeric, character, logical)
- Perform arithmetic and logical operations
- Display the results

2. Data Structures Implementation (BL4 – Analyze)

Write pseudocode to demonstrate advanced data structures in R:

- Create and initialize a vector, list, and matrix
- Perform operations such as indexing, updating, and aggregation
- Compare the structures based on usage

3. Matrix and Array Operations (BL4 – Analyze)

Design pseudocode to perform operations on matrices and arrays:

- Create a matrix and a 3D array
- Perform addition, multiplication (matrix), and element-wise operations
- Display the results and analyze differences

4. Data Frame Construction and Manipulation (BL4 – Analyze))

A dataset needs to be organized efficiently.

Write pseudocode to:

- Create a data frame with multiple columns (numeric and character)
- Add, delete, and modify columns
- Perform basic summary operations

5. Functions and Logical Operations (BL4 – Analyze)

Design pseudocode to implement functions and logical vector operations:

- Create a user-defined function with input parameters
- Perform operations on logical vectors
- Return and display computed results

Code of Conduct:

*I **S THULASISWAR REDDY / 192425369** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

R Programming - Pseudocode Writing Task

Assessment Tool 1 | Course Code: ITA04 | SIMATS Engineering

1. Variables and Data Types

Aim: Declare numeric, character and logical variables, perform arithmetic and logical operations, and display the results.

Pseudocode

```
BEGIN
  // Step 1: Declare variables of different data types
  a      <- 10           // numeric
  b      <- 3            // numeric
  name   <- "R Programming" // character
  is_pass <- TRUE        // logical

  // Step 2: Arithmetic operations
  sum_ab <- a + b
  diff_ab <- a - b
  prod_ab <- a * b
  div_ab <- a / b
  mod_ab <- a MOD b      // %% in R
  pow_ab <- a ^ b

  // Step 3: Logical operations
  gt      <- a > b
  eq      <- a == b
  and_res <- (a > b) AND is_pass
  or_res  <- (a < b) OR is_pass
  not_res <- NOT is_pass

  // Step 4: Display results
  DISPLAY name, class(a), class(name), class(is_pass)
  DISPLAY sum_ab, diff_ab, prod_ab, div_ab, mod_ab, pow_ab
  DISPLAY gt, eq, and_res, or_res, not_res
END
```

Expected Output

Operation	Expression	Result
Data types	class(a), class(name), class(is_pass)	numeric, character, logical
Addition / Subtraction	10 + 3 , 10 - 3	13 , 7
Multiplication / Division	10 * 3 , 10 / 3	30 , 3.333333
Modulus / Power	10 %% 3 , 10 ^ 3	1 , 1000
Comparison	10 > 3 , 10 == 3	TRUE , FALSE
Logical AND / OR / NOT	(10>3) & TRUE , (10<3) TRUE , !TRUE	TRUE , TRUE , FALSE

2. Data Structures Implementation

Aim: Create a vector, list and matrix; perform indexing, updating and aggregation; compare the structures.

Pseudocode

```
BEGIN
// Step 1: Create and initialize
v <- vector(10, 20, 30, 40, 50)           // c() in R
L <- list(name="Asha", age=20, marks=vector(80,90,85))
M <- matrix(1..6, rows=2, cols=3)         // filled column-wise

// Step 2: Indexing
DISPLAY v[2]           // 20
DISPLAY L$name         // "Asha"
DISPLAY M[2,3]         // 6

// Step 3: Updating
v[2] <- 25
L$age <- 21
M[1,1] <- 10

// Step 4: Aggregation
DISPLAY sum(v), mean(v)           // 155, 31
DISPLAY mean(L$marks)             // 85
DISPLAY sum(M), rowSums(M), colSums(M)
END
```

Expected Output

Structure	Operation	Result
Vector v	v[2] -> update to 25 -> sum(v), mean(v)	20 -> v = 10 25 30 40 50 -> 155, 31
List L	L\$name , L\$age (updated) , mean(L\$marks)	"Asha" , 21 , 85
Matrix M	M[2,3] , M[1,1] updated to 10 , sum(M)	6 , 10 , 30
Matrix M	rowSums(M) , colSums(M)	18 12 , 12 7 11

Comparison of the Structures

Feature	Vector	List	Matrix
Data type	Same type only	Mixed types allowed	Same type only
Dimensions	1-D	1-D (can nest)	2-D (rows x columns)
Access	v[i]	L[[i]] or L\$name	M[row, col]
Best used for	Simple sequence of values	Grouping related, different data	Numeric tables, mathematical operations

3. Matrix and Array Operations

Aim: Create a matrix and a 3D array; perform addition, matrix multiplication and element-wise operations; analyze the differences.

Pseudocode

```
BEGIN
  // Step 1: Create two 2x2 matrices
  A <- matrix(1..4, rows=2, cols=2)      // [1 3; 2 4]
  B <- matrix(5..8, rows=2, cols=2)      // [5 7; 6 8]

  // Step 2: Operations on matrices
  add <- A + B                          // addition
  mult <- A %*% B                       // matrix multiplication
  elem <- A * B                         // element-wise multiplication

  // Step 3: Create a 3D array (2 rows, 3 columns, 2 layers)
  arr <- array(1..12, dim=(2, 3, 2))

  // Step 4: Operations on array
  arr_add <- arr + 10                   // element-wise addition
  arr_scale <- arr * 2                  // element-wise multiplication
  DISPLAY arr[2,3,2], sum(arr)

  // Step 5: Display results
  DISPLAY add, mult, elem, arr_add, arr_scale
END
```

Expected Output

Operation	Result
A (columns filled first)	[1 3 ; 2 4]
B	[5 7 ; 6 8]
A + B	[6 10 ; 8 12]
A %*% B (matrix multiplication)	[23 31 ; 34 46]
A * B (element-wise)	[5 21 ; 12 32]
arr[2,3,2] , sum(arr)	12 , 78
arr * 2 (layer 1)	[2 6 10 ; 4 8 12]

Analysis of Differences

Point	Matrix multiplication (%*%)	Element-wise (*)
Rule	Row of A x column of B, then add	Same-position elements multiplied
Example (row 1, col 1)	$1 \times 5 + 3 \times 6 = 23$	$1 \times 5 = 5$
Size condition	Columns of A = rows of B	Both must have the same size
Result	[23 31 ; 34 46]	[5 21 ; 12 32]

4. Data Frame Construction and Manipulation

Aim: Create a data frame with numeric and character columns; add, delete and modify columns; perform summary operations.

Pseudocode

```
BEGIN
// Step 1: Create data frame
df <- data.frame(
  Name = ("Asha", "Ravi", "Meena", "Kiran"), // character
  Age  = (20, 21, 19, 20), // numeric
  Marks = (85, 72, 90, 68)) // numeric

// Step 2: Add a new column
FOR each row i in df
  IF df$Marks[i] >= 80 THEN df$Grade[i] <- "A"
  ELSE df$Grade[i] <- "B"
END FOR // ifelse() does this in one line in R

// Step 3: Modify a column value
df$Marks[2] <- 75 // Ravi: 72 -> 75

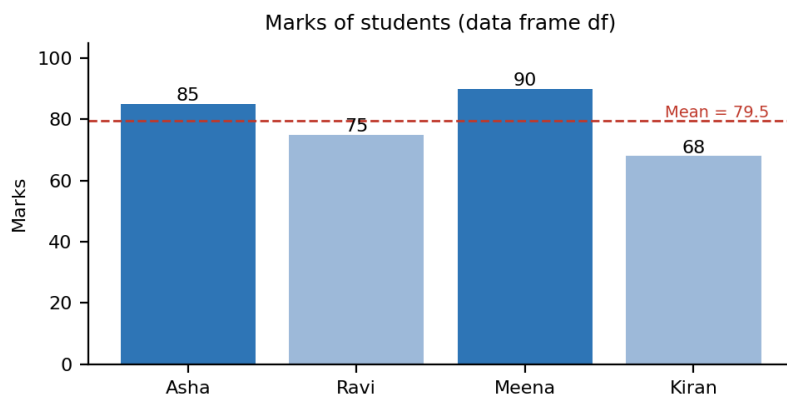
// Step 4: Delete a column
df$Age <- NULL

// Step 5: Summary operations
DISPLAY nrow(df), ncol(df)
DISPLAY mean(df$Marks), median(df$Marks), max(df$Marks), min(df$Marks)
DISPLAY summary(df)
END
```

Expected Output (final data frame)

Name	Marks	Grade
Asha	85	A
Ravi	75	B
Meena	90	A
Kiran	68	B

Summary	nrow	ncol	mean	median	max	min
Value	4	3	79.5	80	90	68



5. Functions and Logical Operations

Aim: Create a user-defined function with parameters, perform logical vector operations, and return and display the results.

Pseudocode

```
FUNCTION analyze_marks(marks, cutoff)
  passed <- marks >= cutoff           // logical vector
  count <- sum(passed)                // TRUE counts as 1
  percent <- count / length(marks) * 100
  RETURN list(passed, count, percent)
END FUNCTION

BEGIN
  // Step 1: Input
  marks <- (85, 72, 90, 68, 55)
  cutoff <- 70

  // Step 2: Call the function
  result <- analyze_marks(marks, cutoff)
  DISPLAY result$passed, result$count, result$percent

  // Step 3: Logical vector operations
  x <- (TRUE, FALSE, TRUE)
  y <- (TRUE, TRUE, FALSE)
  DISPLAY x AND y, x OR y, NOT x
  DISPLAY any(result$passed), all(result$passed), which(result$passed)
END
```

Expected Output

Item	Result
passed	TRUE TRUE TRUE FALSE FALSE
count , percent	3 , 60
x & y , x y , !x	TRUE FALSE FALSE , TRUE TRUE TRUE , FALSE TRUE FALSE
any() , all() , which()	TRUE , FALSE , 1 2 3

